

Group Members:

Muhammad Shehryar Atiq (B23110006115)

Ayan Nasir (B23110006019)

Minimax Tic-Tac-Toe AI

Artificial Intelligence Semester Project Report

Project category: Adversarial search and game theory

Implementation: Python 3, HTML, CSS, and JavaScript

Core algorithm: Minimax with alpha-beta pruning

1. Abstract

This project implements an intelligent Tic-Tac-Toe opponent that applies adversarial search to select moves. The system models the board as a finite, deterministic, two-player, zero-sum game. On Hard difficulty, the AI searches the complete reachable game tree with Minimax and alpha-beta pruning. Therefore, against any sequence of legal player moves, its result is always a win or draw.

The project also acts as an interactive AI laboratory. In addition to playing the game, a student can observe how many nodes were explored, how many branches were pruned, the evaluation returned by

the search, and a short explanation of the selected move.

2. Problem statement

Build an interactive 3×3 Tic-Tac-Toe system in which a human can play against an AI agent. The AI must make decisions using Minimax, correctly identify terminal states, and remain unbeatable at the highest difficulty. The system should make the algorithm understandable through a usable visual interface.

3. Objectives

1. Represent a Tic-Tac-Toe state in a form suitable for search.
2. Generate every legal successor state.
3. Detect horizontal, vertical, and diagonal wins plus draws.
4. Implement Minimax for rational adversarial decision-making.
5. Optimize the search with alpha-beta pruning.
6. Compare random, mixed, and optimal decision policies.
7. Expose search behavior through live metrics and explanations.
8. Verify the "never loses" claim with automated exhaustive tests.

4. Functional requirements

Core

- Interactive 3×3 board with X and O
- Player vs AI mode
- Human-first and AI-first options
- X or O selection
- Winner and draw detection
- Current-turn status
- Restart at any time

Intermediate

- Easy, Medium, and Hard difficulty
- Persistent scoreboard
- Move history with board coordinate and move time
- Winning-cell highlight
- Synthesized click, win, and draw sounds
- Local Player vs Player mode

Advanced

- Alpha-beta pruning
- AI thinking animation
- Per-move timer
- Responsive light and dark themes
- Persistent statistics dashboard
- Saved match history
- Undo in Player vs Player mode
- Explored-node, pruned-branch, search-time, and evaluation metrics
- Explainable AI move descriptions

5. System architecture

Browser interface

- ├ Board and interaction controller
- ├ Score, timer, history, themes, and sounds
- └ POST /api/ai-move



Python HTTP server

- ├ Request and board validation
- └ Game engine
 - ├ Terminal-state evaluation
 - └ Move generation

- └─ Minimax
- └─ Alpha-beta pruning
- └─ Move explanation

The browser owns presentation and current round state. The API is stateless: each request sends the current board, AI symbol, and difficulty. This design keeps the AI engine independent, deterministic in its evaluation, and easy to test.

6. Board representation

The nine board squares are stored in a one-dimensional list:

```
Index:   0 | 1 | 2
         ---+---+---
          3 | 4 | 5
         ---+---+---
          6 | 7 | 8
```

Each position contains "X", "O", or `None`. A flat list makes copying, indexing, move generation, API serialization, and winning-line checks simple.

The eight possible winning lines are stored as index triplets:

- Rows: (0, 1, 2), (3, 4, 5), (6, 7, 8)
- Columns: (0, 3, 6), (1, 4, 7), (2, 5, 8)
- Diagonals: (0, 4, 8), (2, 4, 6)

7. Minimax algorithm

Minimax assumes both participants choose rationally:

- The maximizing player is the AI and wants the highest utility.
- The minimizing player is the opponent and wants the lowest utility.

- The recursion ends at a win, loss, or draw.

The evaluation function is depth-aware:

```
AI win:           +10 - depth
Opponent win:    depth - 10
Draw:            0
```

This scoring makes the AI prefer a quicker win. If a loss is unavoidable, it prefers the path that delays it.

Pseudocode

```
MINIMAX(board, depth, maximizing, alpha, beta)
    if board is terminal
        return EVALUATE(board, depth)

    if maximizing
        best = -infinity
        for every legal move
            best = max(best, MINIMAX(nextBoard, depth + 1,
                                     not maximizing, alpha, beta))
            alpha = max(alpha, best)
            if beta <= alpha
                stop exploring this branch
        return best

    best = +infinity
    for every legal move
        best = min(best, MINIMAX(nextBoard, depth + 1,
                                 maximizing, alpha, beta))
        beta = min(beta, best)
        if beta <= alpha
            stop exploring this branch
    return best
```

8. Alpha-beta pruning

Alpha stores the best result already guaranteed to the maximizing player. Beta stores the best result already guaranteed to the minimizing player. Once `beta <= alpha`, the remaining children cannot affect the final selection and are skipped.

Pruning does not change the move selected by Minimax. It only reduces the number of nodes evaluated. Searching center, corners, and then edges improves the likelihood of finding useful bounds early.

9. Difficulty design

Difficulty	Decision policy	Expected behavior
Easy	Random legal move	Can be defeated easily
Medium	70% optimal Minimax, 30% random evaluated move	Competitive but fallible
Hard	Best Minimax score with alpha-beta pruning	Cannot lose

When multiple moves share the same optimal score, Hard mode may select any one of them. The decision remains mathematically optimal while repeat games feel less mechanical.

10. Explainable decision rules

After selecting a move, the engine classifies its most understandable tactical reason:

1. completed a winning line;
2. blocked an immediate opposing win;
3. created a fork;
4. controlled the center;
5. selected a corner;

6. protected a guaranteed draw; or
7. followed the strongest evaluated path.

The explanation is supplementary. Minimax's score, rather than the explanation rule, is responsible for the actual decision.

11. Complexity

For branching factor b and search depth d :

- Minimax time: $O(b^d)$
- Depth-first recursion space: $O(d)$
- Ideal alpha-beta time with excellent ordering: approximately $O(b^{(d/2)})$

Tic-Tac-Toe is small enough to search completely. The same concepts extend to larger adversarial problems, although those usually require depth limits and heuristic evaluation.

12. Testing strategy and evidence

The automated suite covers:

- row, column, and diagonal victory detection;
- draw detection;
- legal move generation;
- invalid API board rejection;
- taking an immediate winning move;
- blocking an immediate loss;
- the theoretical empty-board draw under perfect play;
- every possible legal human path when the human starts; and
- every possible legal human path when the AI starts.

Run:

```
python -m unittest -v
```

Expected result: all tests pass. The exhaustive path tests fail immediately if any human branch can defeat Hard AI.

13. User-interface considerations

- Large square controls and keyboard keys 1–9 support efficient interaction.
- Color is reinforced by symbols and text, rather than being the only cue.
- Live regions announce AI thinking and game status.
- Responsive breakpoints support desktop, tablet, and mobile layouts.
- Reduced-motion preferences disable non-essential animation.
- Match information persists in browser local storage without transmitting personal data.

14. Limitations

- The app supports one local browser session and does not provide online multiplayer.
- A 3×3 solved game has a small search space; larger games require heuristics.
- Medium difficulty intentionally introduces randomness and is not reproducible unless random seeding is added.
- Browser storage is device-specific and can be cleared by the user.

15. Future enhancements

- Generalize the engine to an N×N board and K-in-a-row win condition.
- Add iterative deepening and heuristic evaluation for larger boards.
- Visualize the selected branch of the game tree.
- Compare Minimax with Monte Carlo Tree Search.

- Add online multiplayer with a server-authoritative game state.
- Export experiment results as CSV for algorithm comparison.

16. Conclusion

The project satisfies its main AI objective: applying adversarial search to make rational decisions in a competitive environment. Hard mode is not merely described as unbeatable; its behavior is verified over all possible legal human paths. The metrics and explanation layer connect the underlying theory to observable behavior, making the application suitable both as a playable system and as a semester demonstration.